

SOFTWARE DESIGN SPECIFICATION (SDS)

INDUSTRIAL BOTTLE PACKAGING LINE AUTOMATION

Document Information

Item	Description
Document Title	Software Design Specification (SDS)
Project	Industrial Bottle Packaging Line Automation
Document Number	SDS-001
Version	1.0
Prepared By	Aditya Katariya
PLC Platform	Allen-Bradley MicroLogix 1100
Programming Software	RSLogix 500
Programming Language	Ladder Logic (LAD)
Date	July 2026

Revision History

Version	Date	Description
1.0	July 2026	Initial Release

Table of Contents

1. Introduction
2. Software Overview
3. Programming Philosophy
4. PLC Hardware Configuration
5. Software Architecture
6. Program Execution Sequence

7. Memory Organization
 8. Program File Description
 9. Data Files
 10. Timer and Counter Usage
 11. Fault Handling Logic
 12. Simulation Philosophy
 13. Coding Standards
 14. Future Software Expansion
-

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) describes the software architecture, program organization, execution sequence, and control philosophy of the Industrial Bottle Packaging Line Automation project.

The document provides a detailed description of how the PLC software has been structured to achieve modularity, readability, maintainability, and ease of troubleshooting.

Unlike the Functional Design Specification (FDS), which defines **what** the machine is required to do, this document explains **how the PLC software has been organized to implement those requirements**.

1.2 Scope

This document covers the PLC software developed for the Industrial Bottle Packaging Line using RSLogix 500.

The document includes:

- PLC program architecture
- Program execution sequence
- Program files
- Memory organization
- Data files
- Timer usage

- Counter usage
- Internal control philosophy
- Simulation strategy
- Alarm handling logic
- Software design principles

This document does not describe the detailed machine operation, which is documented separately in the Sequence of Operation (SOO).

1.3 Software Objectives

The PLC software has been developed with the following objectives:

- Maintain a modular software structure.
 - Separate machine functions into independent routines.
 - Improve program readability.
 - Simplify troubleshooting.
 - Reduce software complexity.
 - Allow future expansion.
 - Support HMI communication.
 - Support simulation without physical hardware.
-

2. Software Overview

The Bottle Packaging Line is controlled by a single Allen-Bradley MicroLogix 1100 PLC programmed using Ladder Logic within RSLogix 500.

The software follows a modular programming approach in which the Main routine performs initialization and sequentially executes individual program routines, each responsible for a specific machine function.

This structure minimizes code duplication and improves software organization while simplifying future modifications.

The PLC continuously executes the application during every scan cycle, ensuring that sensor inputs, machine states, production data, and alarm conditions remain continuously updated.

3. Programming Philosophy

The software has been designed using modular industrial programming principles.

Instead of implementing all machine logic within one large routine, the application has been divided into multiple functional routines.

Each routine performs one specific responsibility.

Examples include:

- Input processing
- Machine mode control
- Sensor processing
- Device control
- Alarm management
- Production statistics
- Simulation
- Cycle reset

This programming philosophy provides several advantages:

- Easier troubleshooting
- Improved readability
- Better software organization
- Faster future modifications
- Reduced programming errors
- Simplified maintenance

4. PLC Hardware Configuration

The software has been developed for the Allen-Bradley MicroLogix 1100 PLC.

The PLC provides:

- Digital Inputs
- Digital Outputs
- Internal Memory

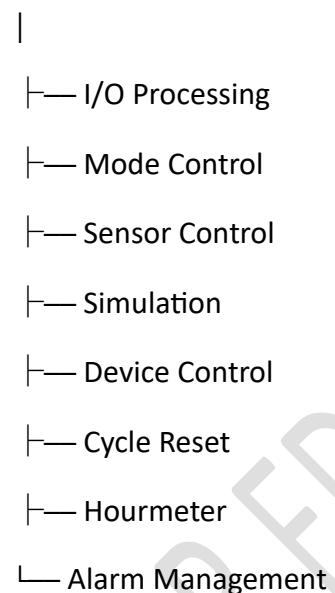
- Timers
- Counters
- Integer Files
- Floating Point Data
- Status Files

Communication with the HMI is achieved through RSLinx Classic during simulation using RSLogix Emulate 500.

5. Software Architecture

The software follows a hierarchical architecture.

MAIN



The Main routine acts as the central program controller.

Each PLC scan begins in the Main routine, which executes every application routine in a predefined sequence.

This architecture ensures deterministic program execution and prevents unexpected behaviour caused by uncontrolled routine execution.

6. Program Execution Sequence

6.1 Overview

The PLC executes the application program continuously during every scan cycle. The Main routine serves as the entry point of the software and is responsible for calling all application routines in a predefined sequence.

The execution order has been selected to ensure that machine inputs are updated before control decisions are made, outputs are controlled correctly, production statistics are maintained, and alarm conditions are evaluated before the scan is completed.

This sequential execution provides deterministic program behaviour and ensures consistent machine operation.

6.2 PLC Scan Cycle

During each PLC scan, the controller performs the following operations:

1. Reads all physical and internal input values.
2. Executes the Main program.
3. Calls each application routine in sequence.
4. Updates internal memory values.
5. Writes output values.
6. Repeats the scan continuously while the controller remains in RUN mode.

This process allows the machine to continuously respond to changing process conditions.

6.3 Routine Execution Order

The Main routine executes the application routines in the following order:

Execution Order	Program File	Primary Function
1	MAIN	Program initialization and routine execution
2	I/O	Input processing and output mapping
3	MODE CTRL	Machine operating mode management
4	SNS IN CTR	Sensor processing and validation

Execution Order	Program File	Primary Function
5	SIMULATION	Manual simulation and fault generation
6	DEVICE CTR	Control of machine actuators
7	CYCLE RST	Reset of temporary machine states
8	HOURLMETER	Runtime and production monitoring
9	ALARMS	Alarm detection and management

This execution sequence remains consistent during every PLC scan.

6.4 First Scan Initialization

When the PLC transitions from Program Mode to Run Mode, the First Scan bit becomes active.

During this initialization phase, the software performs several startup tasks to prepare the system for operation.

Typical initialization activities include:

- Clearing temporary variables.
- Resetting internal status bits.
- Initializing production values.
- Setting the machine to OFF Mode.
- Preparing the control program for automatic operation.

This initialization ensures that the machine always starts from a known and safe operating condition.

6.5 Continuous Program Execution

After completion of the First Scan routine, the PLC enters continuous operation.

During every scan cycle:

- Machine inputs are evaluated.
- Machine operating mode is determined.
- Sensor conditions are processed.
- Simulation logic is executed.

- Output devices are controlled.
- Production statistics are updated.
- Active alarms are evaluated.

The PLC repeats this process continuously until the machine is stopped or the controller is switched out of RUN mode.

7. Memory Organization

7.1 Overview

The PLC software uses several memory areas to store process information, machine states, production statistics, timers, counters, and operator settings.

These memory areas allow the PLC to retain information between scan cycles and coordinate machine operation.

The memory structure has been organized to improve readability and simplify troubleshooting.

7.2 Digital Inputs

Digital input memory is used to store the status of field devices connected to the PLC.

Typical digital inputs include:

- Start Push Button
- Stop Push Button
- Emergency Stop
- Bottle Detection Sensors
- Cap Presence Sensor
- Home Position Sensors
- Alarm Reset Push Button

These inputs provide the PLC with the information required to determine the current machine condition.

7.3 Digital Outputs

Digital output memory controls the field devices operated by the PLC.

Typical outputs include:

- Conveyor Motors
- Filling Valve
- Capping Actuator
- Sorting / Reject Actuator
- Alarm Indicator

The PLC updates these outputs continuously according to the active machine state.

7.4 Internal Memory

Internal memory bits are used to manage machine operation and store intermediate logic conditions.

Typical applications include:

- Machine operating modes
- Process sequencing
- Simulation enable signals
- Fault status
- Bottle tracking
- Alarm status
- Reset conditions

Using internal memory reduces software complexity and improves program organization.

7.5 Timers

Timers are used throughout the application to control process timing and machine sequencing.

Typical timer applications include:

- Filling duration
- Conveyor delays

- Actuator movement timing
- Fault detection delays
- Alarm delays

Timers ensure that machine operations occur for the correct duration before the PLC advances to the next process stage.

7.6 Counters

Counters are used to record production data during machine operation.

The software maintains counters for:

- Total bottles processed.
- Total rejected bottles.
- Runtime calculations.
- Production performance monitoring.

Counter values are continuously updated during production and displayed on the HMI.

7.7 Analog Data

Analog memory is used to represent the bottle filling level.

The PLC processes the analog input value and compares it with the operator-configured preset filling level to determine when the filling operation should stop.

7.8 Data Integrity

The software has been organized to ensure that internal variables remain synchronized throughout the production cycle.

Temporary variables are reset during the Cycle Reset routine after completion of each bottle cycle.

This prevents residual data from affecting subsequent production cycles and improves overall system reliability.

8.2 MAIN Routine

Purpose

The MAIN routine is the primary program file of the Bottle Packaging Line Automation project.

It acts as the central controller of the PLC application by performing system initialization during startup and executing every application routine during each PLC scan.

No machine operation is controlled directly within the MAIN routine. Instead, its primary responsibility is to coordinate program execution by calling the appropriate subroutines in the required sequence.

Responsibilities

The MAIN routine performs the following functions:

- Initializes the PLC during the First Scan.
- Clears internal variables where required.
- Places the machine in a safe startup condition.
- Sets the initial machine operating mode.
- Executes all application routines in the correct order.
- Maintains continuous program execution during normal operation.

First Scan Initialization

When the PLC enters RUN mode, the First Scan instruction becomes active.

During this stage, the MAIN routine performs system initialization to ensure that the machine starts from a known operating condition.

Initialization activities include:

- Resetting temporary memory bits.
- Clearing production variables where required.
- Initializing machine status.
- Setting the machine to OFF Mode.
- Preparing the application for automatic operation.

This initialization prevents unintended machine operation immediately after power-up or program download.

Routine Execution

After initialization has been completed, the MAIN routine continuously executes all application routines during every PLC scan.

The execution order is fixed to ensure deterministic program behaviour and proper synchronization between machine functions.

The application routines are executed in the following order:

1. I/O Processing
2. Mode Control
3. Sensor Control
4. Simulation
5. Device Control
6. Cycle Reset
7. Hourmeter
8. Alarm Management

This sequence is repeated continuously while the PLC remains in RUN mode.

Inputs

The MAIN routine receives information from:

- PLC First Scan status.
- Internal control bits.
- Machine operating status.
- System memory.

It does not process field devices directly.

Outputs

The MAIN routine does not directly control field devices.

Its outputs consist of:

- System initialization.
 - Execution of application routines.
 - Overall program coordination.
-

Design Philosophy

The MAIN routine has intentionally been kept simple.

By avoiding machine control logic within the MAIN routine, the software remains:

- Easy to understand.
- Easy to troubleshoot.
- Easier to modify.
- Highly modular.

All process-specific logic is delegated to dedicated routines, allowing each section of the software to perform a single clearly defined function.

Benefits of the MAIN Routine Design

The adopted design provides several engineering advantages:

- Clear program structure.
- Deterministic execution.
- Simplified debugging.
- Better software maintenance.
- Reduced programming errors.
- Easy addition of future routines without major software restructuring.

The MAIN routine therefore serves as the foundation of the PLC software architecture by coordinating all machine functions while maintaining a structured and organized application.

8.3 I/O Routine

Purpose

The I/O routine manages the interface between the PLC program and the simulated field devices. It is responsible for acquiring input signals, updating internal variables, and controlling the digital outputs used throughout the Bottle Packaging Line.

By separating input and output processing into a dedicated routine, the software maintains a clear distinction between field device communication and process control logic.

Responsibilities

The I/O routine performs the following functions:

- Reads digital input signals from the simulated field devices.
 - Updates internal memory based on the status of push buttons, sensors, and other inputs.
 - Controls digital outputs for conveyors, actuators, valves, and indicators.
 - Maps physical I/O to the variables used by the application.
 - Provides a centralized location for all field device communication.
-

Inputs Processed

Typical inputs handled by this routine include:

- Start Push Button
 - Stop Push Button
 - Emergency Stop
 - Bottle Detection Sensors
 - Cap Presence Sensor
 - Home Position Sensors
 - Alarm Reset
 - Alarm Silence
-

Outputs Controlled

The routine updates outputs associated with:

- Conveyor motors
 - Filling valve
 - Capping actuator
 - Sorting/Reject actuator
 - Alarm indicator
 - Other simulated field devices used in the project
-

Design Philosophy

Keeping all input and output processing in a dedicated routine improves software organization and makes hardware changes easier to implement. If a field device is added, removed, or reassigned, the required modifications can generally be confined to this routine without affecting the remainder of the application.

Interaction with Other Routines

The I/O routine provides the information required by the remaining application routines. Sensor states processed here are used by Mode Control, Sensor Control, Device Control, and Alarm Management to determine machine behaviour.

The routine continuously evaluates the current machine condition before allowing a transition to the next operating mode.

Typical transition criteria include:

- Operator commands.
- Bottle position.
- Sensor feedback.
- Filling completion.
- Capping completion.
- Sorting completion.
- Active fault conditions.

Transitions occur only when all required conditions have been verified.

Fault Handling

When a machine fault is detected, the MODE CTRL routine modifies machine operation according to the severity of the fault.

Depending on the fault type, the routine may:

- Prevent mode transition.
 - Hold the current operating state.
 - Return the machine to OFF Mode.
 - Prevent machine restart until the fault has been cleared.
-

Interaction with Other Routines

The MODE CTRL routine exchanges information with several application routines:

- Receives sensor status from SNS IN CTR.
- Receives operator commands from I/O.
- Sends operating mode information to DEVICE CTR.
- Coordinates fault status with ALARMS.

This interaction ensures that all machine functions operate according to the current machine state.

Design Philosophy

The machine control philosophy is based on a state-driven approach.

Each production stage is completed before the PLC transitions to the next operating mode. This approach simplifies troubleshooting, improves software readability, and ensures predictable machine behaviour.

8.5 SNS IN CTR Routine

Purpose

The SNS IN CTR routine is responsible for processing and validating all sensor signals used throughout the Bottle Packaging Line.

Rather than allowing individual routines to access sensor inputs directly, this routine centralizes sensor processing to improve software organization and ensure consistent interpretation of field signals.

Responsibilities

The SNS IN CTR routine performs the following functions:

- Reads bottle detection sensors.
 - Processes cap presence detection.
 - Validates actuator home position sensors.
 - Updates internal sensor status bits.
 - Provides processed sensor information to the remaining application routines.
-

Sensor Processing

The routine continuously evaluates all process sensors during every PLC scan.

Typical sensor functions include:

- Bottle detection.
- Filling station confirmation.
- Capping station confirmation.
- Cap presence verification.
- Home position confirmation.
- Sorting station detection.

Each sensor is processed before the machine operating mode is updated.

Interaction with Other Routines

The processed sensor information is supplied to:

- MODE CTRL
- DEVICE CTR
- SIMULATION
- ALARMS

This centralized processing ensures consistent sensor interpretation throughout the application.

Design Philosophy

Separating sensor processing from machine control improves software maintainability.

Any future modification to sensor behaviour can generally be implemented within this routine without affecting the remainder of the PLC program.

8.6 SIMULATION Routine

Purpose

The SIMULATION routine allows the Bottle Packaging Line to operate without physical hardware by generating simulated machine inputs and controlled fault conditions.

This routine has been developed to support software testing, demonstration, HMI validation, and educational use.

Responsibilities

The SIMULATION routine performs the following functions:

- Simulates bottle movement.
- Simulates sensor operation.
- Generates bottle detection signals.
- Simulates filling level behaviour.
- Simulates cap presence.

- Generates manual fault conditions.
 - Supports complete machine operation using RSLogix Emulate 500.
-

Fault Simulation

The routine provides the capability to manually simulate the following faults through the HMI:

- Leak Bottle Fault
- Cap Missing Fault
- Capping Arm Retract Jam
- Capping Arm Complete Jam
- Sorting Arm Retract Jam
- Sorting Arm Complete Jam

These simulations allow operators to verify PLC logic, alarm generation, HMI indication, and fault recovery procedures without requiring physical machine failures.

Simulation Philosophy

Simulation is intended solely for software verification and demonstration.

During simulation, the PLC responds to generated signals in the same manner as it would respond to actual field devices, allowing realistic testing of machine operation and fault handling.

Interaction with Other Routines

The SIMULATION routine exchanges information with:

- I/O
- SNS IN CTR
- MODE CTRL
- DEVICE CTR
- ALARMS

This interaction ensures that simulated inputs are processed using the same control logic as real hardware inputs.

Benefits

The simulation routine provides several advantages:

- Enables complete software testing without physical equipment.
- Simplifies debugging during software development.
- Supports HMI validation.
- Demonstrates machine operation for educational purposes.
- Allows safe testing of fault recovery procedures.

The inclusion of a dedicated simulation routine significantly enhances the flexibility of the project and supports efficient software development and validation.

8.7 DEVICE CTR Routine

Purpose

The DEVICE CTR routine is responsible for controlling all machine actuators used throughout the Bottle Packaging Line. Based on the current operating mode, processed sensor information, and machine status, this routine energizes or de-energizes the required outputs to perform the production process.

This routine forms the interface between the PLC decision-making logic and the physical machine outputs.

Responsibilities

The DEVICE CTR routine performs the following functions:

- Controls conveyor operation.
 - Controls the filling valve.
 - Controls the capping actuator.
 - Controls the sorting/reject actuator.
 - Operates output devices according to the active machine state.
 - Prevents output conflicts during machine operation.
-

Conveyor Control

The conveyor is controlled according to the active production stage.

The conveyor operates to:

- Transport bottles between processing stations.
- Stop bottles accurately at processing locations.
- Resume bottle movement after completion of each operation.

The conveyor remains stopped whenever processing is taking place.

Filling Valve Control

The filling valve is controlled during the filling operation.

The valve is energized only when:

- A bottle is correctly positioned.
- The machine is in Filling Mode.
- No filling-related fault is active.

The valve is de-energized immediately after the preset filling level has been achieved.

Capping Actuator Control

The capping actuator is controlled to perform one complete operating cycle.

The routine:

- Extends the actuator.
- Applies the cap.
- Retracts the actuator.
- Confirms return to the home position.

The production sequence does not continue until the capping cycle has been completed successfully.

Sorting and Reject Control

The reject actuator operates only when a bottle has been classified as defective.

The routine removes bottles affected by:

- Leak Bottle Fault
- Cap Missing Fault

Acceptable bottles continue through the production line without reject operation.

Design Philosophy

Output control has been centralized within one routine to improve software organization.

This approach provides:

- Easier troubleshooting.
 - Simplified maintenance.
 - Improved readability.
 - Reduced duplication of output logic.
 - Easier future expansion.
-

8.8 CYCLE RST Routine

Purpose

The CYCLE RST routine prepares the Bottle Packaging Line for the next production cycle by clearing temporary machine information after completion of the current bottle.

The routine ensures that information associated with one production cycle does not affect subsequent bottles.

Responsibilities

The routine performs the following functions:

- Clears temporary machine status bits.
- Resets sequence flags.
- Clears temporary bottle tracking information.

- Returns internal variables to their normal operating condition.
 - Prepares the machine for the next production cycle.
-

Cycle Completion

After bottle processing has been completed, the PLC executes the Cycle Reset routine before returning to Transit Mode.

This ensures that every production cycle begins from a known operating condition.

Benefits

The dedicated Cycle Reset routine:

- Prevents residual data.
 - Improves software reliability.
 - Simplifies troubleshooting.
 - Supports repeatable machine operation.
-

8.9 HOURMETER Routine

Purpose

The HOURMETER routine continuously calculates machine runtime and maintains production-related statistical information.

The collected data is displayed on the HMI to provide operators with real-time production information.

Responsibilities

The HOURMETER routine performs the following functions:

- Calculates machine runtime.
- Updates production counters.
- Calculates bottles per hour.
- Maintains production statistics.
- Provides runtime information for the HMI.

Runtime Monitoring

The routine continuously accumulates machine operating time while the system is running.

Runtime information assists operators in monitoring machine utilization and provides basic maintenance reference information.

Production Statistics

The following production values are maintained:

- Total bottles processed.
- Total rejected bottles.
- Bottles per hour.
- Machine runtime.

Production statistics are updated every minute and displayed on the HMI.

Design Philosophy

Separating production statistics from machine control logic improves software organization and allows future production reporting features to be added without affecting the main control routines.

8.10 ALARMS Routine

Purpose

The ALARMS routine continuously monitors the Bottle Packaging Line for abnormal operating conditions and manages all alarm-related functions within the PLC.

The routine is responsible for detecting machine faults, generating alarms, maintaining fault status, and supporting operator recovery procedures.

Responsibilities

The ALARMS routine performs the following functions:

- Detects abnormal operating conditions.

- Generates machine alarms.
 - Maintains active fault status.
 - Supports alarm acknowledgement.
 - Supports alarm reset.
 - Communicates alarm information to the HMI.
-

Alarm Detection

The routine continuously evaluates machine conditions during every PLC scan.

The implemented alarms include:

- Leak Bottle Fault
- Cap Missing Fault
- Capping Arm Retract Jam
- Capping Arm Complete Jam
- Sorting Arm Retract Jam
- Sorting Arm Complete Jam
- Emergency Stop

When any monitored condition becomes abnormal, the corresponding alarm is generated automatically.

Alarm Processing

Once an alarm has been detected, the routine performs the required machine response according to the type of fault.

Typical actions include:

- Rejecting the affected bottle.
 - Stopping machine operation.
 - Preventing continuation of the production cycle.
 - Displaying the alarm on the HMI.
 - Requiring operator intervention.
-

Alarm Reset

Following correction of the fault condition, the operator performs the Alarm Reset procedure.

The ALARMS routine then:

- Clears the active alarm.
 - Resets internal fault bits.
 - Restores normal machine operation.
 - Allows production to restart after a new Start command has been issued.
-

Design Philosophy

The alarm system has been developed to provide reliable fault detection while maintaining a clear separation between machine control and alarm management.

Centralizing alarm processing within a dedicated routine simplifies maintenance, improves troubleshooting, and allows future alarm conditions to be added with minimal impact on the remainder of the software.

8.11 Summary of Program Files

Program File	Primary Function
MAIN	Initializes the system and executes all application routines.
I/O	Processes field inputs and controls digital outputs.
MODE CTRL	Controls machine operating modes and state transitions.
SNS IN CTR	Processes and validates sensor information.
SIMULATION	Simulates machine operation and fault conditions.
DEVICE CTR	Controls conveyors, actuators, and process outputs.
CYCLE RST	Resets temporary machine states after each production cycle.
HOURLMETER	Calculates runtime and production statistics.
ALARMS	Detects, manages, and clears machine alarms.

9. Data File Organization

9.1 Overview

The PLC software utilizes various data files to store process information, machine states, production statistics, timers, counters, and operator-configurable values. These data files provide the working memory required for machine operation and allow information to be retained and shared between different program routines.

The data organization follows a structured approach to improve software readability, simplify troubleshooting, and support future expansion.

9.2 Input Data

Input data represents information received from the field devices connected to the PLC.

Typical input information includes:

- Start Push Button
- Stop Push Button
- Emergency Stop
- Bottle Detection Sensors
- Filling Position Sensor
- Cap Presence Sensor
- Home Position Sensors
- Alarm Reset
- Alarm Silence

These inputs provide the current status of the machine and are continuously monitored during every PLC scan.

9.3 Output Data

Output data represents the commands sent by the PLC to control the machine actuators.

The PLC controls the following primary outputs:

- Conveyor Motors
- Filling Valve

- Capping Actuator
- Sorting / Reject Actuator
- Alarm Indicator

Output values are updated continuously according to the current machine operating mode.

9.4 Internal Memory

Internal memory is used to store temporary information required for machine operation.

Typical applications include:

- Machine operating modes.
- Process sequence flags.
- Bottle tracking information.
- Fault status.
- Simulation status.
- Alarm status.
- System initialization.
- Internal interlocks.

Internal memory allows different program routines to exchange information while maintaining a clear separation between software functions.

9.5 Integer Data

Integer data files are used to store whole-number values required by the application.

Typical integer values include:

- Bottle counters.
- Reject counters.
- Production statistics.
- Runtime values.
- Configuration parameters.
- Production rate calculations.

These values are available to the HMI for display and monitoring.

9.6 Timer Data

Timers are used throughout the application to control machine timing and sequencing.

Typical timer applications include:

- Filling duration.
- Conveyor movement delay.
- Actuator extension time.
- Actuator retraction time.
- Alarm delay.
- Runtime calculations.

Timers ensure that operations occur for the required duration before the PLC advances to the next production stage.

9.7 Counter Data

Counters are used to maintain production-related information.

The application maintains counters for:

- Total bottles processed.
- Total rejected bottles.
- Production monitoring.
- Runtime calculations.

Counter values are continuously updated and displayed on the HMI.

9.8 Analog Data

Analog data is used to represent the filling level of the bottle.

The PLC compares the measured filling value with the operator-defined preset filling level to determine when the filling process has been completed.

This comparison allows the filling operation to stop automatically once the required level has been achieved.

9.9 Data Sharing Between Routines

All application routines exchange information through shared PLC memory.

Examples include:

- Sensor status processed by the **SNS IN CTR** routine.
- Machine mode information generated by **MODE CTRL**.
- Output commands produced by **DEVICE CTR**.
- Runtime information maintained by **HOURLMETER**.
- Alarm status generated by **ALARMS**.

This centralized data-sharing approach ensures consistent information throughout the application.

9.10 Data Integrity

To ensure reliable operation, temporary variables are reset after completion of each production cycle.

The software prevents previous production data from affecting subsequent bottles by:

- Clearing temporary status bits.
- Resetting sequence flags.
- Initializing cycle-specific variables.
- Updating production counters only after successful cycle completion.

This approach improves software stability and ensures repeatable machine behaviour.

10. Coding Standards and Programming Practices

10.1 Programming Philosophy

The PLC software has been developed using a modular programming approach in which each routine performs a single, clearly defined function.

This philosophy improves software organization and simplifies future maintenance.

10.2 Routine Organization

Each ladder routine has been designed with a specific responsibility.

Examples include:

- Machine initialization.
- Input and output processing.
- Machine mode control.
- Sensor processing.
- Simulation.
- Device control.
- Alarm management.
- Runtime calculation.

This organization minimizes code duplication and improves program readability.

10.3 Naming Convention

Descriptive names have been used throughout the application to improve readability and simplify troubleshooting.

Program files, internal variables, and machine functions are named according to their intended purpose wherever practical.

10.4 Program Readability

The PLC software has been organized to allow engineers to quickly understand the overall control strategy.

The software structure emphasizes:

- Logical routine separation.
- Sequential program execution.
- Clear responsibility for each routine.
- Consistent software organization.

This approach reduces the time required for troubleshooting and future modifications.

10.5 Software Maintainability

The modular architecture allows individual routines to be modified without affecting unrelated parts of the application.

Future functionality can be added by introducing new routines or extending existing routines while preserving the overall software structure.

10.6 Simulation Support

The software has been designed to operate within a simulated environment using RSLogix Emulate 500.

Dedicated simulation logic allows testing of machine behaviour, HMI functionality, and fault recovery procedures without requiring physical hardware.

This approach accelerates software development and simplifies functional verification.

10.7 Error Prevention

The software incorporates structured control logic to reduce the likelihood of incorrect machine operation.

Programming practices adopted within the project include:

- Sequential machine control.
- Dedicated alarm processing.
- Controlled state transitions.
- Centralized output control.
- Separation of machine control and HMI functions.

These practices improve software reliability and support consistent machine behaviour.

11. Software Testing and Validation

11.1 Purpose

Software testing was carried out to verify that the PLC program performs the intended machine functions and responds correctly to normal operating conditions, simulated faults, and operator commands.

Testing was conducted using **RSLogix Emulate 500** in combination with **Wonderware InduSoft Web Studio (AVEVA Edge)** to simulate machine operation and validate communication between the PLC and HMI.

11.2 Functional Testing

Each major machine function was individually tested before evaluating complete system operation.

The following functions were verified:

- Machine startup and shutdown.
- Bottle transportation.
- Filling operation.
- Leak bottle handling.
- Capping operation.
- Cap presence verification.
- Sorting and reject operation.
- Production statistics.
- Alarm generation.
- Emergency Stop functionality.

Successful completion of these tests confirmed that the PLC software performs the required machine operations.

11.3 Simulation Testing

The simulation routine was used to verify machine behaviour without requiring physical hardware.

Simulation testing included:

- Bottle movement.
- Sensor activation.
- Filling process.
- Conveyor operation.

- Actuator operation.
- Fault simulation.
- Alarm handling.
- Production monitoring.

The simulation environment allowed complete validation of the software during development.

11.4 HMI Validation

The HMI was tested to verify correct communication with the PLC.

The following items were validated:

- Start and Stop commands.
- Configuration settings.
- Production counters.
- Runtime display.
- Bottles per hour calculation.
- Alarm display.
- Alarm reset.
- User authentication.
- Manual fault simulation.

All HMI functions operated correctly with the PLC software throughout testing.

11.5 Fault Testing

Each implemented fault was tested individually to verify correct PLC response.

The following fault conditions were evaluated:

- Leak Bottle Fault.
- Cap Missing Fault.
- Capping Arm Retract Jam.
- Capping Arm Complete Jam.

- Sorting Arm Retract Jam.
- Sorting Arm Complete Jam.
- Emergency Stop.

Testing confirmed that alarms were generated correctly and that the machine responded according to the intended control philosophy.

11.6 Performance Verification

Production monitoring features were verified during continuous machine operation.

The following parameters were evaluated:

- Runtime calculation.
- Bottle counter accuracy.
- Reject counter accuracy.
- Bottles per hour calculation.
- Alarm response.
- Machine state transitions.

The software maintained stable operation throughout testing.

12. Software Limitations

The software has been developed as a Proof of Concept (PoC) to demonstrate industrial automation principles.

The following features are outside the scope of the current implementation:

- Physical hardware validation.
- Servo motion control.
- Variable Speed Drive (VFD) integration.
- Industrial communication protocols.
- Vision inspection systems.
- Barcode or QR code tracking.
- SQL database integration.
- SCADA integration.

- Manufacturing Execution System (MES).
- Automatic recipe management.
- Predictive maintenance.

These features may be incorporated in future software versions.

13. Future Software Enhancements

The modular architecture adopted within this project provides a strong foundation for future expansion.

Potential software enhancements include:

- Multiple bottle size recipes.
- Dynamic filling level adjustment.
- Automatic conveyor speed control.
- Vision-based quality inspection.
- Automatic reject confirmation.
- SQL database production logging.
- SCADA connectivity.
- OPC UA communication.
- MQTT integration for IIoT applications.
- Predictive maintenance functions.
- OEE (Overall Equipment Effectiveness) monitoring.
- Energy consumption monitoring.
- Remote diagnostics.
- Multi-user access levels.
- Automatic report generation.

The existing modular software structure allows these features to be integrated with minimal changes to the current program architecture.

14. Software Conclusion

The Software Design Specification describes the architecture and implementation philosophy of the PLC software developed for the Industrial Bottle Packaging Line Automation project.

The software has been organized using a modular programming approach in which individual routines perform dedicated functions such as input processing, machine mode control, sensor validation, device control, simulation, runtime calculation, cycle reset, and alarm management.

This architecture improves program readability, simplifies troubleshooting, and supports future software expansion while maintaining reliable machine operation.

The implementation demonstrates industrial programming practices using Allen-Bradley MicroLogix 1100 PLC hardware and RSLogix 500 Ladder Logic programming.

Although developed as a Proof of Concept, the software reflects the structured programming techniques commonly used in industrial automation projects and provides a scalable foundation for future enhancements.

References

The following hardware and software were used during the development of this project:

Hardware

- Allen-Bradley MicroLogix 1100 PLC

Software

- RSLogix 500
- RSLogix Emulate 500
- RSLinx Classic
- Wonderware InduSoft Web Studio (AVEVA Edge)

Engineering Concepts

- Modular PLC Programming
- Sequential Machine Control
- Human-Machine Interface Design
- Industrial Automation Engineering Practices